CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-G1
System Integration and Architecture

System Design Document (SDD)

Project Title: CebuNest

Prepared By: Saligue, Kean Maverick

Version: 6

Date: 2/21/2026

Status: Final

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
1	[02/21/2026]	Kean Maverick Saligue	(Initial Draft): Added the content for the executive Summary, Introduction, and Functional Requirements Specifications	Final
2	[02/27/2026]	Kean Maverick Saligue	Added contents for System Architecture, API Contract and Communication Flow, Database Design, UI/UX Design, and Project Plan.	Final
3	04/03/2026	Kean Maverick Saligue	Refined core API Contracts and established JSON response structures. Added foundational error handling codes and HTTP status standards.	Final
4	04/09/2026	Kean Maverick Saligue	Integrated advanced features into Functional Requirements (MoSCoW), including PayMongo Sandbox payments, Lease Extensions, and Google OAuth login workflows.	Final
5	04/26/2026	Kean Maverick Saligue	Expanded Admin Oversight workflows (User Management, Property Approval, Audit Logs, Global Broadcasting). Added Acceptance Criteria 7 & 8 for Admin controls.	Final
6	5/05/2026	Kean Maverick Saligue	Expanded Database Section. Added lease_extensions, audit_logs, property_edit_requests, and property_types with full column definitions	Final

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY

1.1 Project Overview

CebuNest is a rental management platform designed to streamline the renting process for tenants and property owners in Cebu City. The system allows tenants to browse available properties, submit rental requests, and manage their bookings.

Property owners can create and manage property listings, review rental requests, and monitor payments. The platform follows a three-tier architecture consisting of a Spring Boot backend API, a React web application, and an Android mobile application dedicated exclusively to the tenant view.

1.2 Objectives

1. Develop a fully functional rental management MVP with user authentication, property listings, booking requests, and basic management features.
2. Implement a three-tier architecture using Spring Boot (backend), React (web), and Android (mobile).
3. Create RESTful APIs to enable communication between the backend, web, and mobile applications.
4. Design a clean and responsive user interface that ensures ease of use for both tenants and property owners.
5. Deploy the system in a production-ready setup for real-world use.

1.3 Scope

Included Features:

- User registration and login with JWT authentication
- Role-based access control (Tenant, Owner, and Admin)
- Property listing with search and filter functionality
- Property image upload and file management
- Rental request submission and status management (Pending, Approved, Rejected)
- Owner dashboard for managing property listings and requests
- Sandbox payment gateway integration for fees
- Protected routes and secured API endpoints

- Responsive web interface (React TypeScript)
- Kotlin Android mobile application
- Sending email using SMTP services
- Google OAuth Login
- Locating property location via OpenStreetMap API

Excluded Features:

- Advanced rent billing automation
- Full-scale property analytics dashboard
- Multi-property enterprise management system
- Identity Verification of Owners

1.0 INTRODUCTION

1.1 Purpose

This document serves as the comprehensive design specification for the **CebuNest** Rental Management System. It outlines the system requirements, architectural design, API contracts, database structure, integrations, and implementation roadmap.

The purpose of this document is to guide the development process and ensure that the backend, web, and mobile components integrate seamlessly.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview Project Name: CebuNest
Domain: Rental Management / Property Listing **Primary**
Users:

1. Property Owner
2. Tenant
3. Administrator (Admin)

Problem Statement: Tenants often struggle to find available boarding houses and apartments efficiently, while property owners lack a simple system to manage listings and rental requests digitally. Many processes are manual, unorganized, or communicated through social media only.

Solution: CebuNest provides a streamlined rental platform where tenants can browse properties and submit rental requests, and owners can manage listings and approve or reject requests. The system ensures a consistent experience across web and mobile platforms with secure authentication and structured data management.

2.2 Core User Journeys Journey 1: First-time Tenant Rental Request

1. User visits the web or mobile application.

2. Clicks "Sign Up", creates an account (via email or Google OAuth), and chooses the Tenant role.

3. Logs in successfully.

4. Browses property listings via the interactive map or list view.

5. Views detailed property information (specs, location, images).

6. Submits a rental request specifying the start date and lease duration.

7. Receives an automated email and in-app notification once the owner approves or rejects the request.

8. Upon approval, confirms the reservation and pays the initial fee via the PayMongo sandbox integration.

Journey 2: Returning Tenant

1. User logs in with existing credentials
2. Navigates to "My Rentals" page
3. Views a list of properties they are currently renting
4. Checks detailed information for each rental, including:

    - Rental period or lease duration
    - Payment status (Paid, Pending, Overdue)
    - Upcoming payment due dates
    - Property owner contact information

5. Makes payment if any installment is due
6. Requests a lease extension if desired.
7. Receives confirmation email and/or in-app notification for completed payment
8. Optionally, views past rental history and receipts

9. Submits a property review/rating once the lease is confirmed or completed.

Journey 3: Owner Property Management

1. Owner logs in with Owner credentials.
2. Navigates to the Owner Dashboard to view analytics (revenue, occupancy, ratings).
3. Adds a new property with details, map pinning, and image uploads (subject to Admin approval).
4. Edits property information if needed.
5. Views incoming rental requests and approves or rejects them.
6. Reviews and responds to lease extension requests from active tenants.
7. Verifies payment statuses of properties (Paid, Pending, Overdue).
8. Attempts to toggle visibility . If a property is blocked or rejected by an admin, the visibility toggle button is disabled for the owner, and the UI displays the explicit deactivation/rejection reason provided by the administrator.
9. Attempts to d

Journey 4: Admin – User Management and Property Approval

1. Admin logs in with Admin credentials
2. Navigates to the Admin Dashboard
3. **Manages Users:**
    - Views all registered users (Tenants, Owners, Admins)
    - Creates new user accounts and assigns roles
    - Updates user details or roles
    - Deactivates or removes users if needed
4. Approves or Rejects Properties
5. Admin logs in with Admin credentials.
6. Navigates to the Admin Dashboard.
7. Manages Users:
    - Views all registered users (Tenants, Owners, Admins).
    - Creates new user accounts and assigns roles.
    - Updates user details, emails, or roles.
    - Deactivates or removes users if needed.
8. Approves or Rejects Properties:
    - Views all property listings submitted by Owners that are pending review.
    - Approves listings, changing their status to APPROVED (making them visible).
    - Rejects listings, changing their status to REJECTED, and provides a mandatory reason.
    - Notifications are sent to property owners regarding approval or rejection.
9. System Oversight:
    - Views paginated audit history of actions taken on properties.
    - Broadcasts system-wide in-app notifications to specific roles (Owners, Tenants, or both).
    - Admin can also view historical actions and audit past approvals/rejections

2.3 Feature List (MoSCoW)

MUST HAVE

1. User Registration and Login (Standard Email & Password)
2. Google OAuth Login integration
3. JWT Authentication (stateless, Bearer token)
4. Password Reset flow (Forgot password via 6-digit email OTP)
5. Role-Based Access Control (Tenant, Owner, and Admin)
6. Property Management (Full CRUD for Owners, Override for Admins)
7. Property Image Upload (file stored on Supabase + linked to DB)
8. Interactive Map Integration (OpenStreetMap API for pinning locations)
9. Property Search and Filtering (location, property type, price range)
10. Rental Request Submission (Tenant)
11. Rental Request Approval/Rejection (Owner)
12. Admin User Management (view, create, update role/email/profile, and deactivate user accounts)
13. Admin able to broadcast notifications
14. Admin notification broadcast notification history
15. Rental Request Status Tracking (PENDING, APPROVED, CONFIRMED, REJECTED, TERMINATED, COMPLETED)
16. Sandbox Payment Gateway Integration (PayMongo)
17. Lease Extension Management (Request, Approve, Reject)
18. Payment Status Tracking (Cron jobs for marking overdue payments)
19. SMTP Email Sending (Welcome emails, OTPs, Payment receipts, Status updates)
20. Property rating and review system
21. Role-Based UI (Tenant, Owner, Admin)
22. Property Edit Request Workflow (Owner submits edits for Admin review; property locks to PENDING_EDIT_REVIEW until approved or rejected)
23. Owner Analytics Dashboard (revenue, occupancy rate, monthly revenue chart, request funnel, property ratings)
24. User Avatar Upload (Supabase Storage, available on both web and mobile)
25. User Profile Update (name, phone number, social links — web and mobile)
26. Automated Payment Scheduling (on rental confirmation, monthly installment rows are generated for the full lease duration)
27. Automated Lease Completion (cron job marks leases COMPLETED when the end date passes, frees the property, and notifies both parties — with outstanding balance detection)
28. Automated Overdue Payment Detection (cron job marks PENDING payments OVERDUE when their due date passes, notifies tenant and owner)
29. Lease Expiry Reminders (automated notifications sent to tenant 1, 2, and 3 days before lease end date)

SHOULD HAVE

1. Rental request history page for tenants
2. Image preview before upload
3. Input validation feedback with specific error messages
4. Rate limiting for login attempts
5. Real-time-like request updates via polling (30-second notification intervals)

COULD HAVE

1. WebSocket-based real-time rental updates
2. SMS notification integration
3. AI Chatbot for answering basic tenant inquiries
4. In-app messaging/chat system between Owner and Tenant
5. Virtual 360-degree property tours or video upload support
6. In-app electronic lease agreement signing (e-signatures)
7. Tenant background check or KYC (Know Your Customer) identity verification integration
8. "Favorite" or "Save" property functionality for tenants to revisit later
9. Dark mode UI toggle for both web and mobile applications

WON'T HAVE

1. Automated monthly rent billing system (payments require manual initiation by tenant)
2. Property owner and Admin view in mobile application.
3. Automated maintenance ticketing and repair tracking system
4. Integration with third-party accounting or tax software
5. Automated late fee penalty calculations and automatic ledger charges
6. Cryptocurrency payment support
7. Multi-language support
8. Advanced enterprise analytics dashboard
9. Owner identity verification
10. Email verification on registration
11. Geolocation restriction enforcement at the API level
12. Forgot Password / account recovery flow on Mobile

2.4 Detailed Feature Specifications Feature: User Authentication

Feature: User Authentication & Account Recovery

- **Screens:** Registration, Login, Forgot Password, Reset Password
- **Fields:** Full Name, Email, Phone Number, Password, Confirm Password, Social Links, Role (Tenant or Owner)
- **Validation: V**alid email format, 8-character minimum strong password.
- **API Endpoints:**
    - POST /api/auth/register
    - POST /api/auth/login
    - POST /api/auth/google
    - POST /api/auth/forgot-password
    - POST /api/auth/verify-reset-code

- POST /api/auth/reset-password
- GET /api/auth/me

- **Security: JWT authentication, BCrypt password hashing (Cost: 12), Protected routes.**

Feature: Property Management (Owner and Admin)

- **Screens:** Property Listing, Property Detail, Owner Dashboard, Admin Property Request, Admin Edit Request, Admin Property Management.
- **Display:** Grid layout, Property image carousels, Title, price, location, specs (beds, baths, sqm), active tenant details.
- **Search & Filter:** By location, status, property type, and price range.
- **API Endpoints:**
  - GET /api/properties (Public catalog)
  - GET /api/properties/types
  - GET /api/properties/my (Owner)
  - POST /api/properties (Owner)
  - PUT /api/properties/{id} (Owner)
  - POST /api/properties/{id}/images (Owner)
  - DELETE /api/properties/{id} (Owner)
  - PUT /api/admin/properties/{id}/visibility (Admin)
  - GET /api/properties/{id}
  - GET /api/admin/properties
  - GET /api/admin/properties/{id}
  - PUT /api/admin/properties/{id}
  - POST /api/admin/properties/{id}/images
  - POST /api/properties/{id}/edit-request
  - GET /api/admin/property-edit-requests
  - GET /api/admin/property-edit-requests/{id}
  - PUT /api/admin/property-edit-requests/{id}/decision

Feature: Rental Request & Lease Management (Tenant and Owner)

- **Screens:** Rental Request Form, Request Status Page, Lease Adjustment Modal.
- **Functions:** Submit request (Tenant), Approve/reject request (Owner), Extend/reduce/terminate lease (Owner), Submit extension request (Tenant).
- **Persistence:** Stored in database, linked to tenant and property.
- **API Endpoints:**
  - POST /api/rental-requests
  - GET /api/rental-requests/my
  - PUT /api/rental-requests/{id}/status
  - PUT /api/rental-requests/{id}/lease
  - PUT /api/rental-requests/{id}/terminate
  - POST /api/lease-extensions
  - PUT /api/lease-extensions/{id}/respond
  - GET /api/rental-requests/my/property/{propertyId}

- GET /api/rental-requests/property/{propertyId}
- GET /api/rental-requests/property/{propertyId}/active
- GET /api/lease-extensions/rental/{rentalRequestId}

Feature: Payment Integration (Sandbox)

- **Screens:** Payment Drawer, Transaction Receipt Modal, PayMongo Checkout.
- **Process:** Redirect to payment provider (test mode), record payment result, generate scheduled installment rows, send email receipts.
- **API Endpoints:**
  - POST /api/payments/confirm (Tenant accepts lease, generates schedule)
  - POST /api/payments/{id}/initiate (Generates PayMongo link)
  - GET /api/payments/{id}/verify (Callback from PayMongo)
  - GET /api/payments/request/{requestId}
  - GET /api/payments/{id}/cancel
  - GET /api/payments/{id}/fail

Feature: Admin Dashboard & Platform Oversight

Screens: Admin Dashboard, Pending Property Requests, User Management, Audit Logs, Property Edit Requests, All Properties, Notifications.

- **Functions:** Review rental properties, approve/reject with mandatory reasons, view paginated **audit** history, broadcast global announcements.
- **Persistence:** Status stored as PENDING_REVIEW, APPROVED, REJECTED. Audit Logs stored for historical tracking.
- **API Endpoints:**
  - GET /api/admin/rental-requests (Pending properties)
  - PUT /api/admin/rental-requests/{id}/status
  - GET /api/admin/audit-logs
  - POST /api/admin/notifications/broadcast
  - GET /api/admin/users
  - POST /api/admin/users
  - GET /api/admin/rental-requests/{id}
  - GET /api/admin/notifications/history
  - PUT /api/admin/users/{id}/role
  - PUT /api/admin/users/{id}/active
  - PUT /api/admin/users/{id}/email
  - PUT /api/admin/users/{id}/profile

Feature: User Profile Management

Screens: Profile Page (web and mobile)

- **Functions:** Update name, phone number, social links; upload and update avatar
- **Persistence:** User data persisted to the users table; avatar files stored in Supabase Storage.
- **API Endpoints:**
    - PUT /api/users/{id}
    - POST /api/users/{id}/avatar

Feature: Notification

Screens: Notification Panel (Tenant, Owner, Admin)

- Functions: View all notifications, mark one as read, mark all as read
- API Endpoints:
    - GET /api/notifications
    - PATCH /api/notifications/{id}/read
    - PATCH /api/notifications/read-all

Feature: Property Reviews & Ratings

Screens: Property Detail Page, My Rentals Page

- Functions: Tenant submits a review after lease is confirmed or completed; public view of reviews per property
- API Endpoints:
    - POST /api/property-reviews
    - GET /api/property-reviews/property/{propertyId}

Feature: Owner Analytics Dashboard

Screens: Owner Dashboard

- Functions: View revenue totals, occupancy rate, monthly revenue chart (last 6 months), request funnel by status, per-property ratings breakdown
- API Endpoints:
    - GET /api/analytics/owner

2.5 Acceptance Criteria

AC-1: Successful User Registration

➢Given I am a new user
➢When I enter a valid email and strong password
➢And confirm that the passwords match
➢And click "Create Account"
➢Then my account should be created
➢And my password should be securely hashed
➢And I am automatically redirected to the appropriate dashboard based on my role.

AC-2: Tenant Rental Request Flow

➢ Given I am logged in as a tenant
➢ When I view an available property and click "Request to Rent"
➢ And enter a valid start date and lease duration
➢ And submit the rental request
➢ Then the rental request should be created
➢ And the rental status should be set to **PENDING** with a status of PENDING.
➢ And the property owner should be notified
➢ And I should receive an automated email notification and app notification once the owner approves or rejects the request

---

AC-3: Owner Property Management

➢ Given I am logged in as a property owner
➢When I view a rental request with status PENDING and choose to approve it.
➢Then the rental status should change to CONFIRMED, the payment schedule is generated, the property is marked UNAVAILABLE, all other pending requests for that property are automatically rejected, and the tenant is notified that their lease is now active.

➢And When I choose to reject the request
➢Then the rental status should change to REJECTED
➢And the tenant should be notified via email and app notification
➢And payment should not be allowed

➢And when a tenant submits a Lease Extension Request.
➢Then I can approve or reject the extension, which automatically adjusts the lease duration and generates the required future payment rows if approved.

AC-4: Rental Payment

➢Given I am logged in as a tenant with a CONFIRMED rental request and a generated payment schedule
➢When I initiate payment for an installment
➢And complete the transaction through the PayMongo gateway
➢Then the payment should be validated upon redirect
➢And the payment status of that installment should change to PAID
➢And payments must be made in installment order — a prior unpaid installment blocks the current one
➢And the owner and tenant should receive payment confirmation receipts via email and app notification

---

AC-5: Admin Property Listing Approval

➢ Given I am logged in as an Admin
➢ When a property owner submits a new property listing
➢ Then the listing is marked as PENDING_REVIEW
➢ And When I approve the listing, it becomes AVAILABLE to tenants, and the owner is notified
➢ And When I reject the listing, the status changes to REJECTED.
➢And an Audit Log entry is created for both approval and rejection actions, and the owner is notified.
➢ And I am able to use the Notifications panel to broadcast system-wide alerts to Tenants, Owners, or both.

AC-6: Admin User Management

➤ Given I am logged in as an Admin
➤ When I access the User Management dashboard.
➤ Then I can view all registered users in the system.

➤ And When I create a new user account, the system stores the information securely and assigns the selected role.
➤ And When I update a user's details, the system saves the details successfully.
➤ And When I deactivate a user account, that user can no longer log into the platform.

AC-7: Admin Global Property Management & Override

➤Given I am logged in as an Admin.

➤When I access the "All Properties" management dashboard.

➤Then I can view a list of all properties on the platform, regardless of their current status.

➤And When I edit a property's details (e.g., title, price, location, specs), my changes bypass the owner ownership restriction and save immediately.

➤And I am able to upload or remove images from any property listing on behalf of the owner.

AC-8: Admin Forced Listing Deactivation (Locking)

➤Given I am logged in as an Admin.

➤When I review a published property (status is AVAILABLE) that violates platform policies.

➤And I choose to deactivate the property, providing a mandatory administrative deactivation reason.

➤ Then the property status should change to UNAVAILABLE.

➤ And the property should be locked in an "Admin Disabled" mode, preventing the owner from republishing it themselves.

➤And the owner should receive an automated notification explaining the administrative action.

➤And the deactivation is blocked if the property currently has an active (CONFIRMED) tenant.

➤ And When I reactivate a previously locked property, the admin-disabled flag is cleared, the property status returns to AVAILABLE, and the owner is notified that their listing has been restored.

AC-9: Property Edit Request Workflow

➤Given I am an Owner with a published (AVAILABLE) property

➤When I submit an edit request with updated details

➤Then the property status changes to PENDING_EDIT_REVIEW

➤And the Admin receives the request with a diff showing old vs proposed values

➤And When the Admin approves, the property fields are updated and status restores to its previous state, and I am notified

➤And When the Admin rejects with a mandatory reason, the original values are restored, and I am notified with the rejection reason

➤And An Audit Log entry is created for both approval and rejection

---

AC-10: Automated Lease Lifecycle

➤Given a CONFIRMED rental exists

➤When the lease end date passes (start date + lease duration months)

➤Then the cron job marks the rental as COMPLETED and the property returns to AVAILABLE

➤And both tenant and owner are notified

➤And any unpaid installments are forced to OVERDUE at completion

➤And reminder notifications are sent to the tenant 1, 2, and 3 days before the end date

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- **Average response time:** ≤ 2 seconds for 95% of API requests
- **Maximum response time:** ≤ 5 seconds under high load
- **Web page load time:** ≤ 3 seconds on broadband
- **Mobile app cold start:** ≤ 3 seconds
- **Concurrent users supported:** ≥ 50 simultaneous users
- **Database query execution:** ≤ 500ms for standard queries

3.2 Security Requirements

- **Data transport:** HTTPS for all communications
- **Authentication:** JWT token authentication for all protected endpoints
- **Password storage:** bcrypt hashing (salt rounds = 12)
- **Vulnerability protection:** SQL injection prevention on all input fields
- **Access control:** Role-based access for tenants and owners; admin endpoints restricted
- **Account access control:** deactivated accounts are blocked at login
- **Admin-disabled property lockout:** owners cannot republish properties locked by an administrator

3.3 Compatibility Requirements

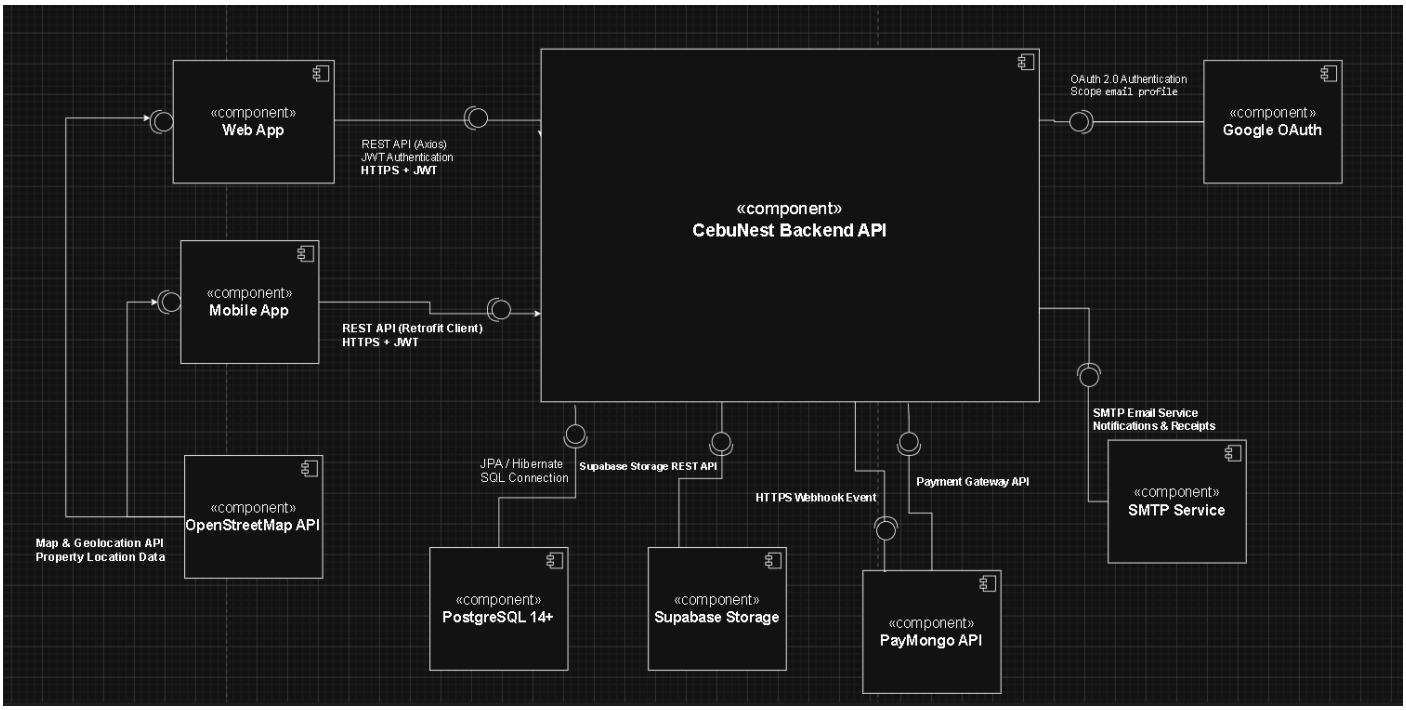
- **Web Browsers:** Chrome, Firefox, Safari, Edge (latest 2 versions)
- **Android:** API Level 34 (Android 14)
- **Screen Sizes:** Mobile (360px+), Tablet (768px+), Desktop (1024px+)
- **Operating Systems:** Windows 10+, macOS 10.15+, Linux Ubuntu 20.04+
- **Backend API:** Compatible with REST clients and mobile Retrofit library
- Mobile app requires internet connection — no offline mode supported

3.4 Usability Requirements

- **Tenant onboarding:** Complete account registration and first rental request within 5 minutes
- **Accessibility:** WCAG 2.1 Level AA compliance for web interfaces
- **Navigation:** Consistent menu and dashboard experience across web and mobile
- **Error handling:** Clear, actionable messages with recovery options
- **Mobile interaction:** Touch targets minimum 44x44px; supports swipe gestures for property browsing

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram



Technology Stack:

- **Backend:** Java 17, Spring Boot 4.0.3, Spring Security, Spring Data JPA, Lombok, Spring Boot Mail Starter, Spring Boot OAuth2 Client, Spring Boot Validation
- **Database:** PostgreSQL 14+ (SupaBase), SupaBase Storage (file storage for property images and avatars)
- **Web Frontend:** React 19, TypeScript, Axios 1.13, React Router DOM 7.13, React Leaflet, @react-oauth/google 0.13
- **Mobile:** Kotlin, Retrofit, Glide, Google Sign-In SDK, Kotlin Coroutines
- **Build Tools:** Maven (Backend), Vite (Web), Gradle (Android)
- **Deployment: Render** (Backend), Vercel (Web), APK (Mobile)

Base URL	https://it342-saligue-cebunest.onrender.com/
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) in Authorization header
Response Structure	{ "success": boolean, "data": object|null, "error": { "code": string, "message": string, "details": object|null }, "timestamp": string }

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

5.2 Endpoint Specifications

Authentication & Account Recovery Endpoints

Description	*API URL*	*HTTP Request Method*	*Format*	*Authentication*	*Request Payload*	*Response Structure*
Register User	/api/auth/register	POST	JSON	None	{ "name": "...", "email": "...", "password": "...", "confirmPassword": "...", "phoneNumber": "...", "role": "TENANT | OWNER", "facebookUrl": "...",	{ "success": true, "data": { "user": {...}, "accessToken": "...", "refreshToken": "..." }, "error": null, "timestamp": "..." }

Description	*API URL*	*HTTP Request Method*	*Format*	*Authentication*	*Request Payload*	*Response Structure*
					"instagramUrl": "...", "twitterUrl": "..." }	
User Login	/api/auth/login	POST	JSON	None	{ "email": "...", "password": "..." }	{ "success": true, "data": { "user": {...}, "accessToken": "...", "refreshToken": "..." }, "error": null, "timestamp": "..." }
Google OAuth	/api/auth/google	POST	JSON	None	{ "token": "<GoogleAccessToken>", "role": "TENANT | OWNER" }	{ "success": true, "data": { "user": {...}, "accessToken": "...", "refreshToken": "..." }, "error": null, "timestamp": "..." }
Forgot Password	/api/auth/forgot-password	POST	JSON	None	{ "email": "..." }	{ "success": true, "data": { "message": "..." }, "error": null, "timestamp": "..." }
Verify OTP	/api/auth/verify-reset-code	POST	JSON	None	{ "email": "...", "code": "123456" }	{ "success": true, "data": { "message": "..." }, "error": null, "timestamp": "..." }
Reset Password	/api/auth/reset-password	POST	JSON	None	{ "email": "...", "code": "123456", "newPassword": "..." }	{ "success": true, "data": { "message": "..." }, "error": null, "timestamp": "..." }

Description	*API URL*	*HTTP Request Method*	*Format*	*Authentication*	*Request Payload*	*Response Structure*
Get Current User	/api/auth/me	GET	JSON	Bearer Token	None	{ "success": true, "data": { "user": {...} }, "error": null, "timestamp": "..." }

Property Endpoints

Description	*API URL*	*HTTP Request Method*	*Format*	*Authentication*	*Request Payload*	*Response Structure*
Get Catalog	/api/properties	GET	JSON	Optional	?search=...&type=...&minPrice=...&maxPrice=...	{ "success": true, "data": { "properties": [...] }, "error": null, "timestamp": "..." }
Get Types	/api/properties/types	GET	JSON	Optional	None	{ "success": true, "data": { "types": [...] }, "error": null, "timestamp": "..." }
Get Details	/api/properties/{id}	GET	JSON	Optional	None	{ "success": true, "data": { "property": {...} }, "error": null, "timestamp": "..." }

Description	*API URL*	*HTTP Request Method*	*Format*	*Authentication*	*Request Payload*	*Response Structure*
Get My Properties	/api/properties/my	GET	JSON	Owner	?status=...	{ "success": true, "data": { "properties": [...] }, "error": null, "timestamp": "..." }
Create Property	/api/properties	POST	JSON	Owner	{ "title": "...", "description": "...", "price": 5000.0, "location": "...", "typeId": 1, "beds": 1, "baths": 1, "sqm": 25 }	{ "success": true, "data": { "property": {...} }, "error": null, "timestamp": "..." }
Update Property	/api/properties/{id}	PUT	JSON	Owner	{ "title": "...", "price": 5000.0, "location": "...", "typeId": 1, "status": "AVAILABLE" }	{ "success": true, "data": { "property": {...} }, "error": null, "timestamp": "..." }
Upload Images	/api/properties/{id}/images	POST	FormData	Owner	MultipartFile[] files	{ "success": true, "data": { "property": {...} }, "error": null, "timestamp": "..." }

Description	*API URL*	*HTTP Request Method*	*Format*	*Authentication*	*Request Payload*	*Response Structure*
Delete Property	/api/properties/{id}	DELETE	JSON	Owner	None	{ "success": true, "data": { "deleted": true, "id": "..." }, "error": null, "timestamp": "..." }

Rental Request Endpoints

Description	**API URL**	**HTTP Request Method**	**Format**	**Authentication**	**Request Payload / Query Params**	**Response Structure**
Submit Request	/api/rental-requests	POST	JSON	Tenant	{ "propertyId": 1, "startDate": "YYYY-MM-DD", "leaseDurationMonths": 6 }	{ "success": true, "data": { "request": {...} }, "error": null, "timestamp": "..." }
Get My Requests	/api/rental-requests/my	GET	JSON	Tenant	None	{ "success": true, "data": { "requests": [...] }, "error": null, "timestamp": "..." }

Description	API URL	HTTP Request Method	Format	Authentication	Request Payload / Query Params	Response Structure
Get Property Reqs	/api/rental-requests/property/{id}	GET	JSON	Owner	None	{ "success": true, "data": { "requests": [...] }, "error": null, "timestamp": "..." }
Get Active Tenant	/api/rental-requests/property/{id}/active	GET	JSON	Owner	None	{ "success": true, "data": { "activeTenant": {...} }, "error": null, "timestamp": "..." }
Update Status	/api/rental-requests/{id}/status	PUT	JSON	Owner	{ "status": "APPROVED | REJECTED" }	{ "success": true, "data": { "request": {...} }, "error": null, "timestamp": "..." }
Adjust Lease	/api/rental-requests/{id}/lease	PUT	JSON	Owner	{ "adjustMonths": 2 } *(Can be negative)*	{ "success": true, "data": { "request": {...} }, "error": null,

Description	API URL	HTTP Request Method	Format	Authentication	Request Payload / Query Params	Response Structure
						"timestamp": "..." }
Terminate Lease	/api/rental-requests/{id}/terminate	PUT	JSON	Owner	None	{ "success": true, "data": { "request": {...} }, "error": null, "timestamp": "..." }
Get My Request for Property	/api/rental-requests/my/property/{propertyId}	GET	JSON	Tenant	None	{ "success": true, "data": { "request": {...} }, "error": null, "timestamp": "..." }

Lease Extension Endpoints

Description	API URL	HTTP Request Method	Format	Authentication	Request Payload / Query Params	Response Structure
Request Extension	/api/lease-extensions	POST	JSON	Tenant	{ "rentalRequestId": 1, "requestedMonths": 2, "reason": "..." }	{ "success": true, "data": { "extensionRequest": {...} }, "error": null, "timestamp": "..." }
Respond Extension	/api/lease-extensions/{id}/respond	PUT	JSON	Owner	{ "decision": "APPROVED | REJECTED" }	{ "success": true, "data": { "extensionRequest": {...} }, "error": null, "timestamp": "..." }
Get Ext. History	/api/lease-extensions/rental/{id}	GET	JSON	Owner/Tenant	None	{ "success": true, "data": { "extensionRequests": [...] }, "error": null, "timestamp": "..." }

Payment Endpoints

Description	API URL	HTTP Request Method	Format	Authentication	Request Payload	Response Structure
Confirm Plan	/api/payments/confirm	POST	JSON	Tenant	{ "requestId": 1 }	{ "success": true, "data": { "request": {...} }, "error": null, "timestamp": "..." }
Get Schedule	/api/payments/request/{id}	GET	JSON	Owner/Tenant	None	{ "success": true, "data": { "payments": [...] }, "error": null, "timestamp": "..." }
Initiate PayMongo	/api/payments/{id}/initiate	POST	JSON	Tenant	None	{ "success": true, "data": { "payment": {"checkoutUrl": "...", "paymongoPaymentId": "..."} }, "error": null, "timestamp": "..." }
Verify Payment	/api/payments/{id}/verify	GET / POST	JSON	Tenant	None	{ "success": true, "data": { "payment": {"status": "PAID", "paidAt": "..."} }, "error": null, "timestamp": "..." }
Cancel Payment	/api/payments/{id}/cancel	GET	JSON	Tenant	None	{ "success": true, "data": { "payment": {...} }, "error": null, "timestamp": "..." }

Description	API URL	HTTP Request Method	Format	Authentication	Request Payload	Response Structure
Mark Failed	/api/payments/{id}/fail	GET	JSON	Tenant	None	{ "success": true, "data": { "payment": {...} }, "error": null, "timestamp": "..." }

Notification & Review Endpoints

Description	API URL	HTTP Request Method	Format	Authentication	Request Payload / Query Params	Response Structure
Get Notifications	/api/notifications	GET	JSON	All Roles	None	{ "success": true, "data": { "notifications": [...] }, "error": null, "timestamp": "..." }
Mark Read (One)	/api/notifications/{id}/read	PATCH	JSON	All Roles	None	{ "success": true, "data": { "notification": {...} }, "error": null, "timestamp": "..." }
Mark Read (All)	/api/notifications/read-all	PATCH	JSON	All Roles	None	{ "success": true, "data": { "message": "..." }, "error": null, "timestamp": "..." }

Description	API URL	HTTP Request Method	Format	Authentication	Request Payload / Query Params	Response Structure
Submit Review	/api/property-reviews	POST	JSON	Tenant	{ "rentalRequestId": 1, "rating": 5, "comment": "..." }	{ "success": true, "data": { "review": {...} }, "error": null, "timestamp": "..." }
Get Reviews	/api/property-reviews/property/{id}	GET	JSON	Optional	None	{ "success": true, "data": { "reviews": [...] }, "error": null, "timestamp": "..." }

Admin Endpoints

Description	API URL	HTTP Request Method	Format	Authentication	Request Payload	Response Structure
Get Pending Reqs	/api/admin/rental-requests	GET	JSON	Admin	None	{ "success": true, "data": { "properties": [...], "count": 1 }, "error": null, "timestamp": "..." }
Review Listing	/api/admin/rental-requests/{id}/status	PUT	JSON	Admin	{ "status": "APPROVED | REJECTED", "reason": "..." }	{ "success": true, "data": { "property": {...} }, "error": null, "timestamp": "..." }

Description	API URL	HTTP Request Method	Format	Authentication	Request Payload	Response Structure
Toggle Visibility	/api/admin/properties/{id}/visibility	PUT	JSON	Admin	{ "reason": "..." }	{ "success": true, "data": { "property": {...} }, "error": null, "timestamp": "..." }
Override Details	/api/admin/properties/{id}	PUT	JSON	Admin	{ "title": "...", "price": 0.0, ... }	{ "success": true, "data": { "property": {...} }, "error": null, "timestamp": "..." }
Get Audit Logs	/api/admin/audit-logs	GET	JSON	Admin	?page=0&size=20	{ "success": true, "data": { "logs": [...], "totalElements": 1, "totalPages": 1, "currentPage": 0 }, "error": null, "timestamp": "..." }
Get All Users	/api/admin/users	GET	JSON	Admin	None	{ "success": true, "data": { "users": [...], "count": 1 }, "error": null, "timestamp": "..." }
Update User Role	/api/admin/users/{id}/role	PUT	JSON	Admin	{ "role": "OWNER | TENANT | ADMIN" }	{ "success": true, "data": { "user": {...} }, "error": null, "timestamp": "..." }
Toggle User Status	/api/admin/users/{id}/active	PUT	JSON	Admin	{ "active": true | false }	{ "success": true, "data": { "user": {...} }, "error": null, "timestamp": "..." }

Description	API URL	HTTP Request Method	Format	Authentication	Request Payload	Response Structure
Global Broadcast	/api/admin/notifications/broadcast	POST	JSON	Admin	{ "type": "...", "message": "...", "targetRoles": ["OWNER", "TENANT"] }	{ "success": true, "data": { "message": "..." }, "error": null, "timestamp": "..." }
Get Broadcast History	/api/admin/notifications/history	GET	JSON	Admin	None	{ "success": true, "data": { "history": [...], "count": 1 }, "error": null, "timestamp": "..." }
Create User	/api/admin/users	POST	JSON	Admin	{ "name": "...", "email": "...", "password": "...", "role": "..." }	{ "success": true, "data": { "user": {...} }, "error": null, "timestamp": "..." }
Update User Email	/api/admin/users/{id}/email	PUT	JSON	Admin	{ "email": "..." }	{ "success": true, "data": { "user": {...} }, "error": null, "timestamp": "..." }
Update User Profile	/api/admin/users/{id}/profile	PUT	JSON	Admin	{ "name": "...", "phoneNumber": "...", "facebookUrl": "...", "instagramUrl": "...", "twitterUrl": "..." }	{ "success": true, "data": { "user": {...} }, "error": null, "timestamp": "..." }

Property Edit Request Endpoints

Description	API URL	HTTP Request Method	Format	Authentication	Request Payload	Response Structure
Submit Edit Request	/api/properties/{id}/edit-request	POST	JSON	Owner	{ "title": "...", "description": "...", "price": 0.0, "location": "...", "typeId": 1, "beds": 1, "baths": 1, "sqm": 25 }	{ "success": true, "data": { "editRequest": {...} }, ... }
Get Pending Edit Requests	/api/admin/property-edit-requests	GET	JSON	Admin	None	{ "success": true, "data": { "editRequests": [...], "count": 1 }, ... }
Get Edit Request Detail	/api/admin/property-edit-requests/{id}	GET	JSON	Admin	None	{ "success": true, "data": { "editRequest": {...} }, ... }
Review Edit Request	/api/admin/property-edit-requests/{id}/decision	PUT	JSON	Admin	{ "decision": "APPROVED | REJECTED", "reason": "..." }	{ "success": true, "data": { "editRequest": {...} }, ... }

User Profile Endpoints

Description	API URL	HTTP Request Method	Format	Authentication	Request Payload	Response Structure
Update Profile	/api/users/{id}	PUT	JSON	Bearer Token	{ "name": "...", "phoneNumber": "...", "facebookUrl": "...", "instagramUrl": "...", "twitterUrl": "..." }	{ "success": true, "data": { "user": {...} }, ... }
Upload Avatar	/api/users/{id}/avatar	POST	FormData	Bearer Token	MultipartFile file (FormData)	{ "success": true, "data": { "avatarUrl": "...", "user": {...} }, ... }

Owner Analytics Endpoints

Description	API URL	HTTP Request Method	Format	Authentication	Request Payload	Response Structure
Get Owner Analytics	/api/analytics/owner	GET	JSON	Owner	None	{ "success": true, "data": { "properties": [...], "requestStats": {...}, "occupancy": {...}, "paymentStats": {...}, "monthlyRevenue": [...], "overallRating": {...},

Description	API URL	HTTP Request Method	Format	Authentication	Request Payload	Response Structure
						"propertyRatings": [...], ... }, ... }

5.3 Error Handling HTTP Status Codes

- 200 OK - Successful request
- 201 Created - Resource created
- 400 Bad Request - Invalid input or validation failure
- 401 Unauthorized - Authentication required/failed
- 403 Forbidden - Insufficient permissions (e.g., non-admin accessing admin route)
- 404 Not Found - Resource doesn't exist
- 409 Conflict - Duplicate resource (e.g., email already registered)
- 500 Internal Server Error - Server error

Error Code Examples

Example 1: Business Logic Error

```
{
  "success": false,
  "data": null,
  "error": {
    "code": "BUSINESS-001",
    "message": "You can only request an extension for an active rental.",
    "details": null
  },
  "timestamp": "2026-05-06T11:20:24.000Z"
}
```


Example 2: Validation Error


```
{
  "success": false,
  "data": null,
  "error": {
    "code": "VALID-001",
    "message": "Requested months must be at least 1.",
    "details": null
  },
  "timestamp": "2026-05-06T11:20:24.000Z"
}
```

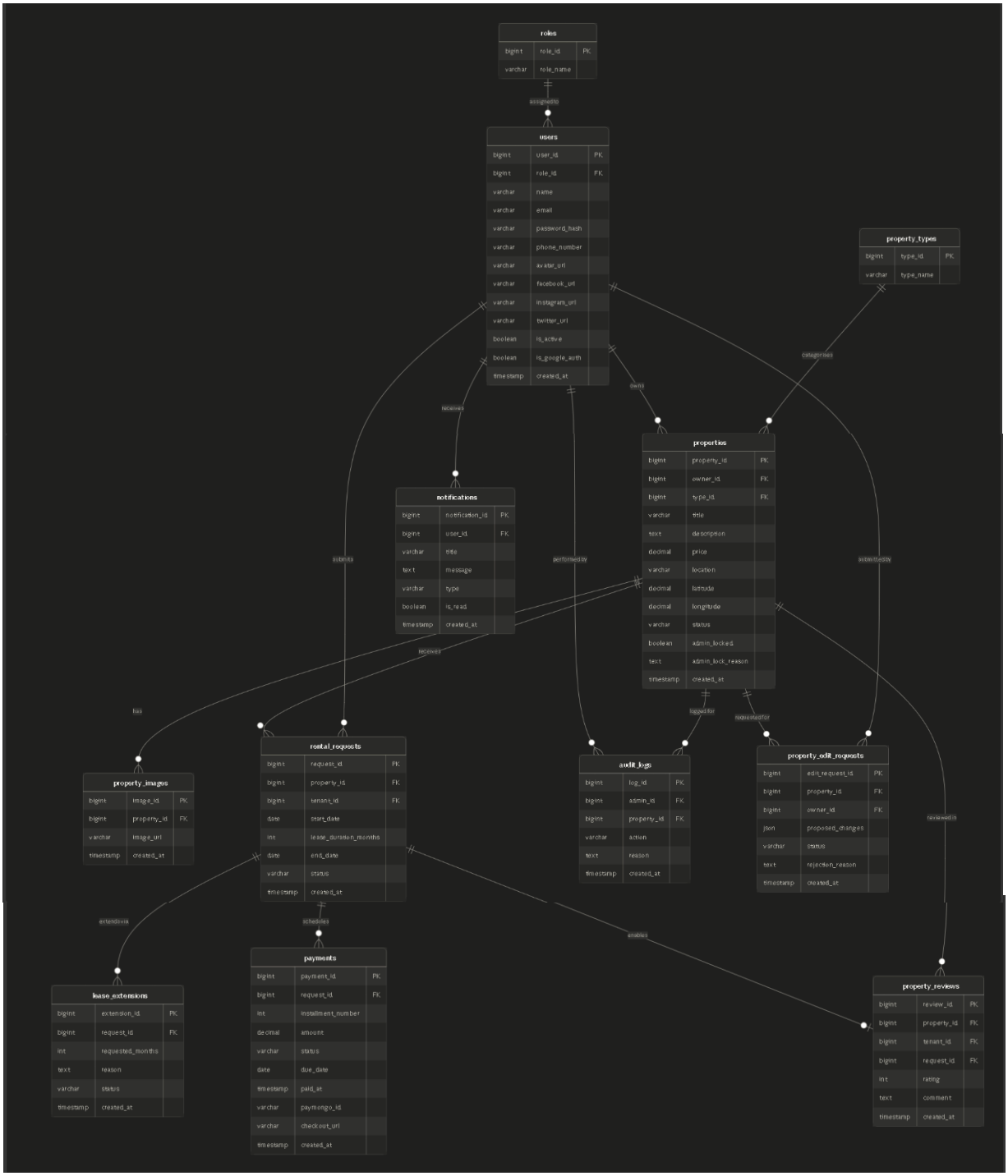

Common Error Codes

- **AUTH-001:** Invalid credentials or not authenticated

- **AUTH-002:** Missing, invalid, or expired token
- **AUTH-003:** Security/Token mismatch (e.g., Code already used)
- **VALID-001:** Validation failed (missing required fields)
- **DB-001:** Resource not found in database
- **BUSINESS-001:** Action violates system business logic (e.g., terminating an already completed lease)
- **SYSTEM-001:** Internal server or third-party API error
- **BUSINESS-002:** Property is no longer available
- **PAYMENT-001:** Payment failed
- **PAYMENT-002:** Payment verification failed

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram



Detailed Relationships:

- **One-to-Many:** Role → Users (Each role can be assigned to multiple users)
- **One-to-Many:** User (Owner) → Properties (An owner can create multiple property listings)
- **One-to-Many:** PropertyType → Properties (A property type can categorise multiple properties)
- **One-to-Many:** Property → PropertyImages (Each property can have multiple images)
- **One-to-Many:** Property → RentalRequests (A property can receive multiple rental requests)
- **One-to-Many:** User (Tenant) → RentalRequests (A tenant can submit multiple rental requests)
- **One-to-Many:** RentalRequest → LeaseExtensions (A rental request can have multiple extension requests submitted over its lifetime)
- **One-to-Many:** RentalRequest → Payments (An approved rental request generates multiple installment payment rows for the full lease duration)
- **One-to-Many:** Property → PropertyReviews (A property can have multiple reviews from different tenants)
- **One-to-Many:** User (Tenant) → PropertyReviews (A tenant can write multiple property reviews across different rentals)
- **One-to-Many:** User → Notifications (A user can receive multiple notifications)
- **One-to-Many:** User (Admin) → AuditLogs (An admin can perform multiple audited actions)
- **One-to-Many:** Property → AuditLogs (A property can have multiple audit log entries over its lifecycle)
- **One-to-Many:** Property → PropertyEditRequests (A property can have multiple edit requests submitted over time)
- **One-to-Many:** User (Owner) → PropertyEditRequests (An owner can submit multiple edit requests across their listings)

Key Tables:

1. **roles** - Defines system roles such as Tenant, Owner, and Admin
2. **users** - User accounts and authentication
3. **property_types** - Categories for property listings
4. **properties** - Property listings created by owners
5. **property_images** - Images associated with properties
6. **rental_requests** - Rental applications submitted by tenants
7. **lease_extensions** - Extension requests submitted by tenants for active rentals
8. payments - Installment payment rows generated upon rental confirmation

9. **property_reviews** - Reviews and ratings given by tenants
10. **notifications** - System notifications sent to users
11. **audit_logs** - Admin actions performed on properties
12. **property_edit_requests** - Property edit submissions pending admin review

Table Structure Summary:

- **roles:** role_id, role_name
- **users:** user_id, role_id, name, email, phone_number, password_hash, avatar_url, facebook_url, instagram_url, twitter_url, is_active, is_google_auth, created_at
- property_types: type_id, type_name
- **properties:** property_id, owner_id, type_id, title, description, price, location, latitude, longitude, status, admin_locked, admin_lock_reason, created_at
- **property_images**: property_image_id, property_id, image_url, created_at
- rental_requests: rental_request_id, property_id, tenant_id, start_date, lease_duration_months, end_date, status, created_at
- **lease_extensions**: extension_id, request_id, requested_months, reason, status, created_at
- **payments:** payment_id, request_id, installment_number, amount, status, due_date, paid_at, paymongo_id, checkout_url, created_at
- **property_reviews**: property_review_id, property_id, tenant_id, request_id, rating, comment, created_at
- **notifications:** notification_id, user_id, title, message, type, is_read, created_at
- **audit_logs**: log_id, admin_id, property_id, action, reason, created_at
- **property_edit_requests**: edit_request_id, property_id, owner_id, proposed_changes, status, rejection_reason, created_at

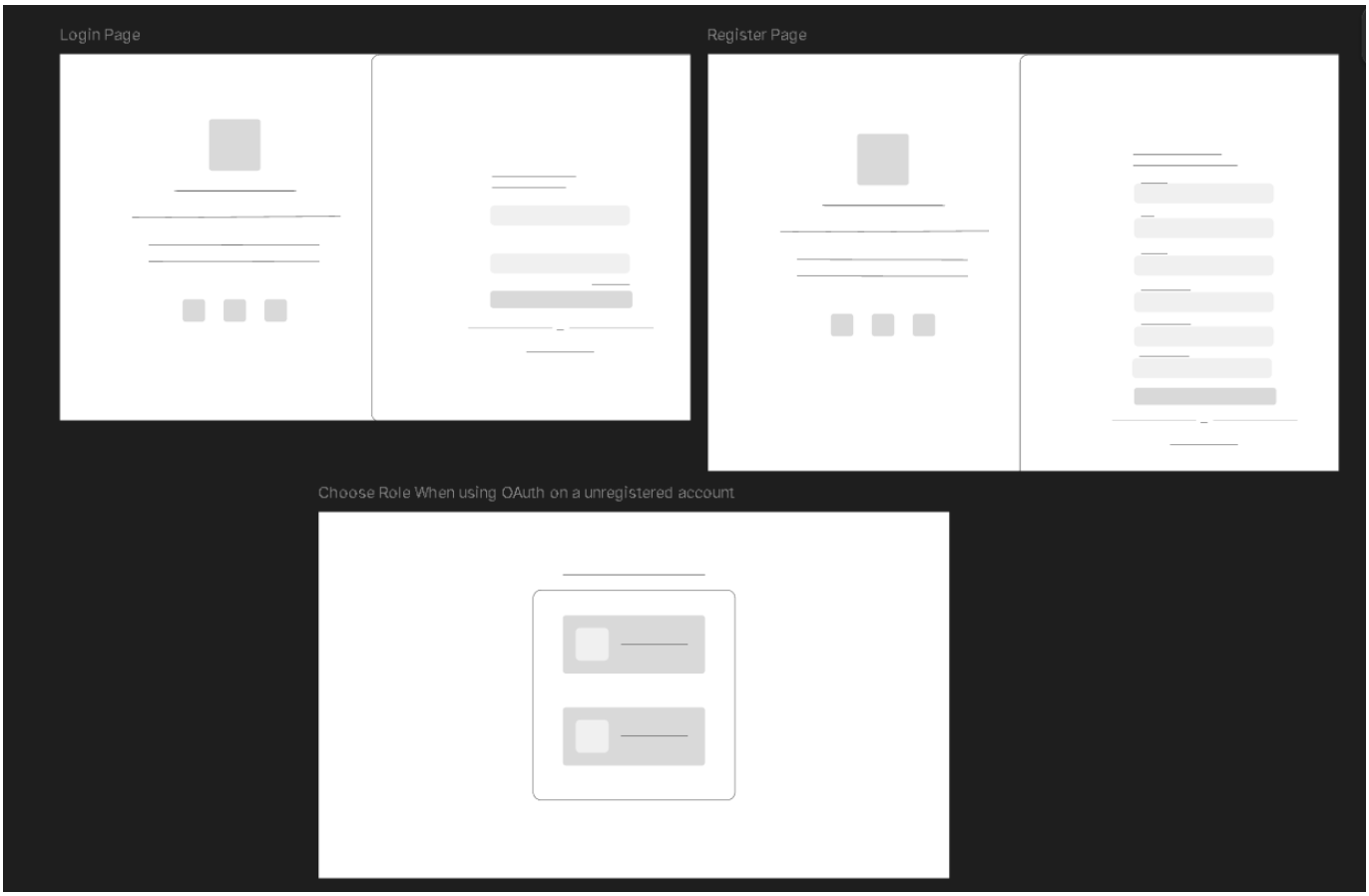
7.0 UI/UX DESIGN

7.1 Web Application Wireframes

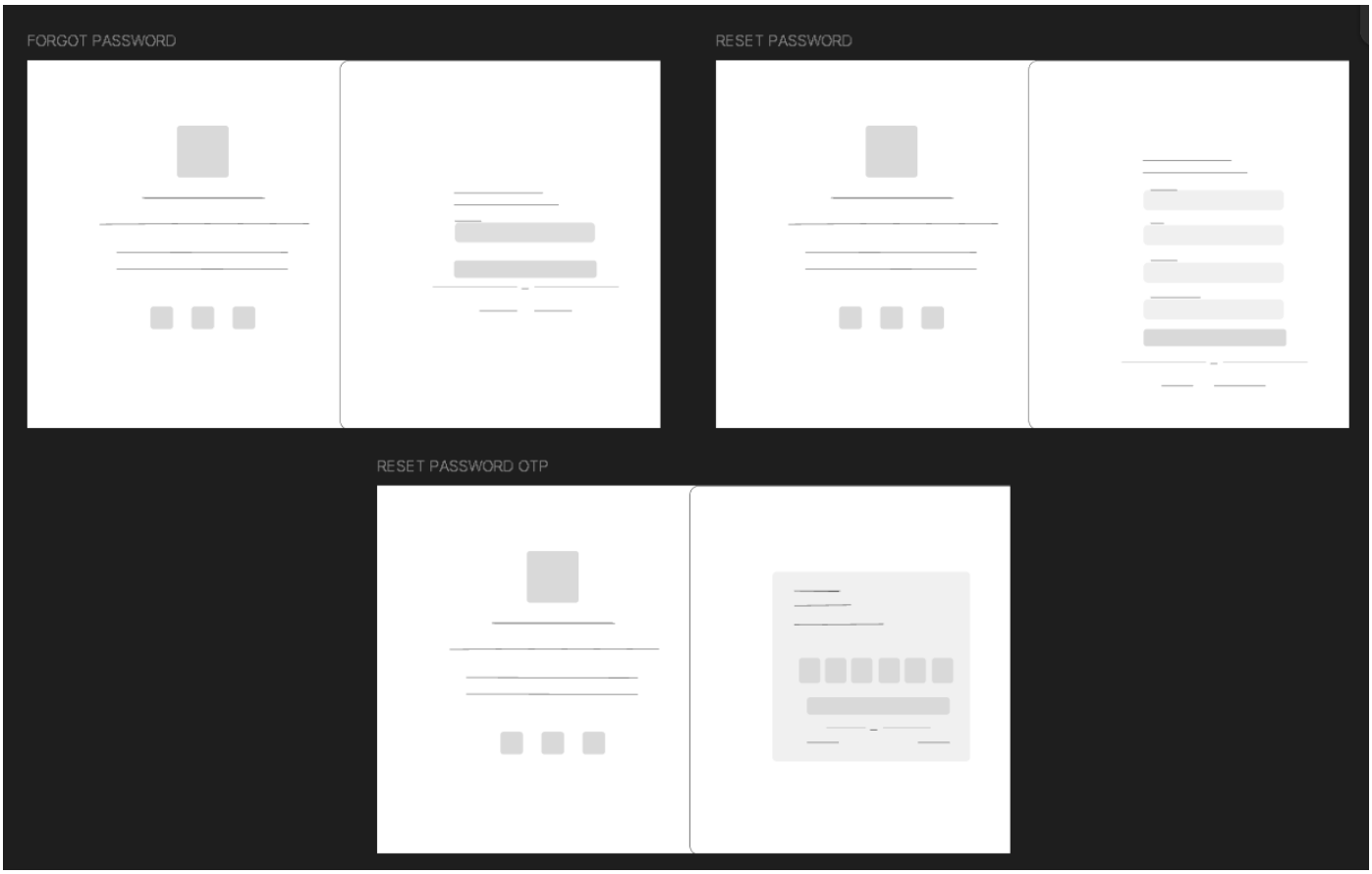
The web version of CebuNest will support tenant screens, admin screens and owner screens.

Figma Link: https://www.figma.com/design/sxZKvg8tHCKc7PYrDJG9ke/CebuNest-Wireframe?node-id=0-1&t=ekJzqTjnVcs5lALf-1

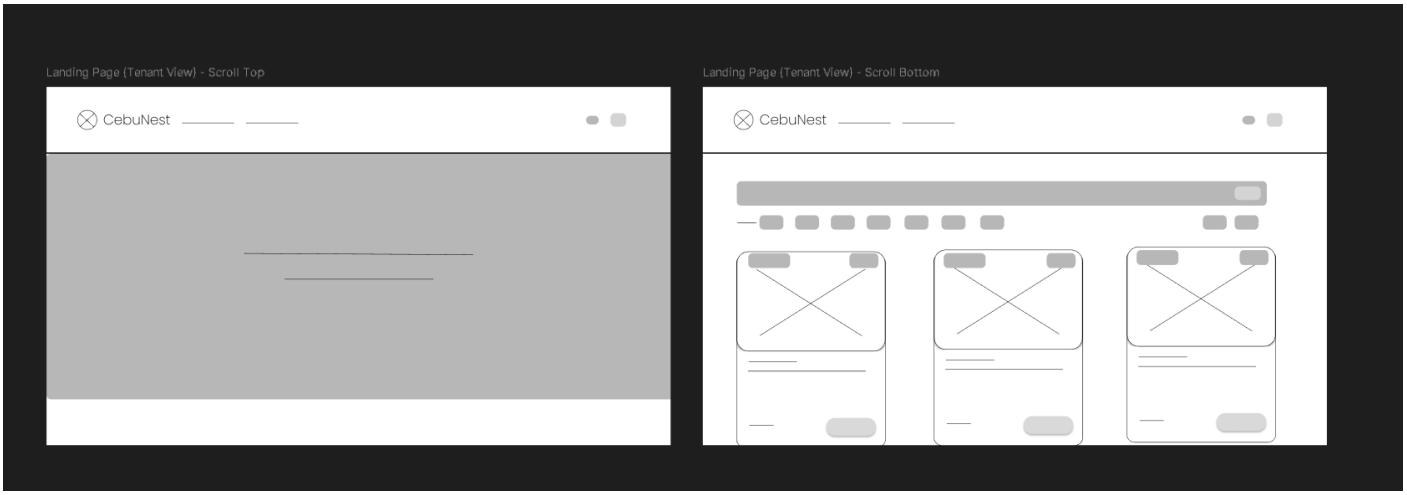
Login and Register Page



Forgot Password Page



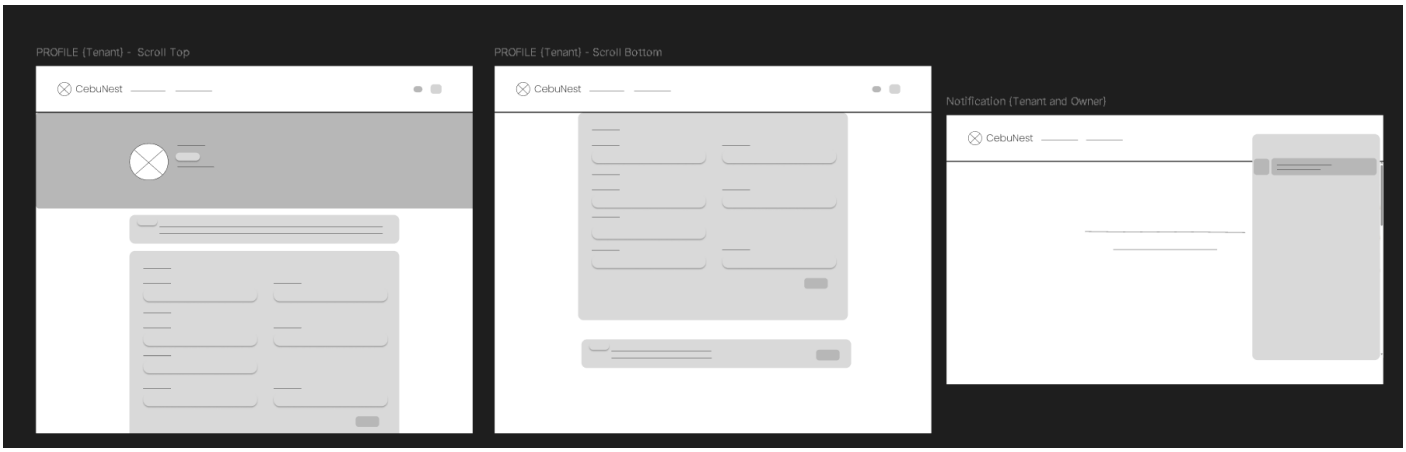
Landing Page (Tenant)



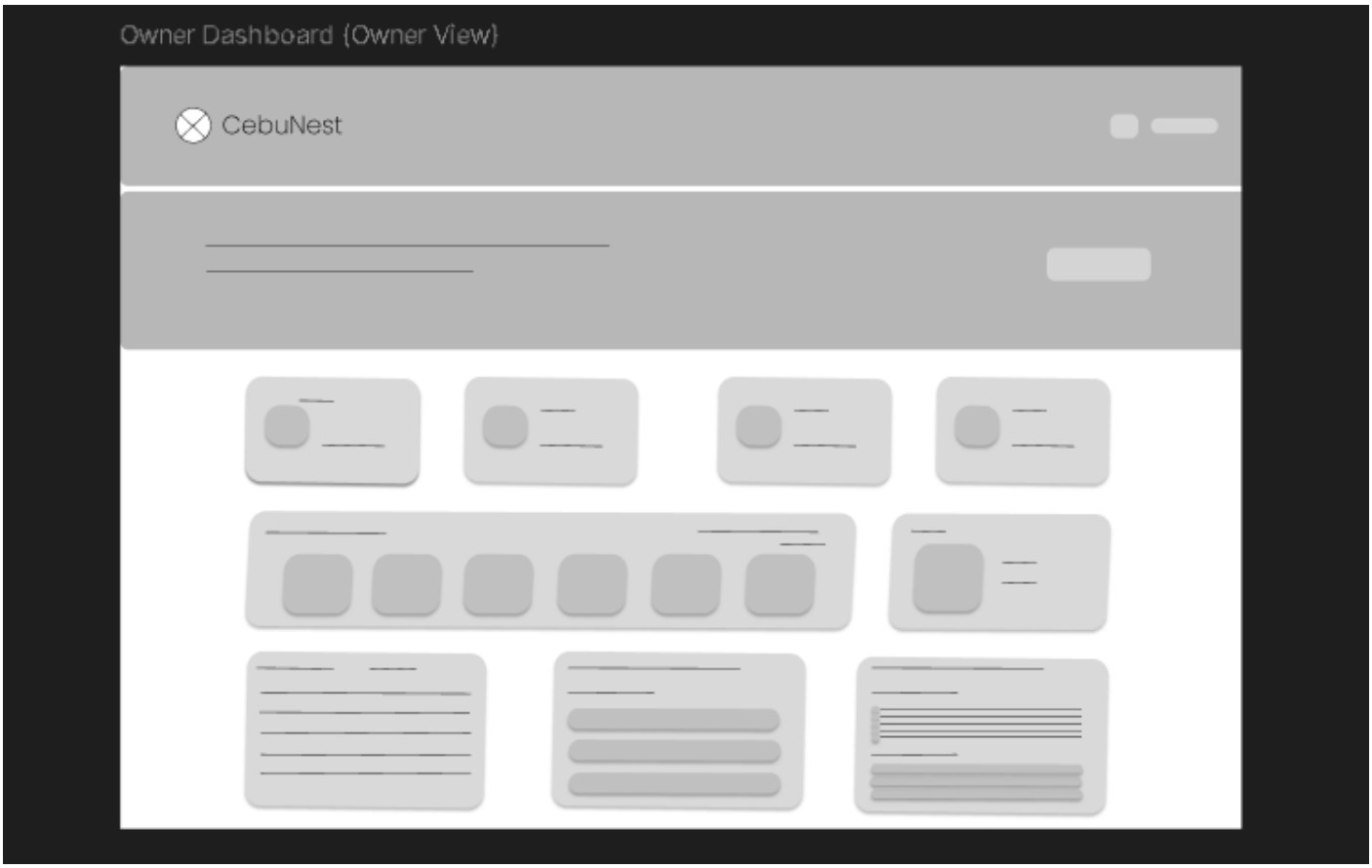
Renting Property Pages Flow (Tenant)

My Rentals Pages (Tenant)

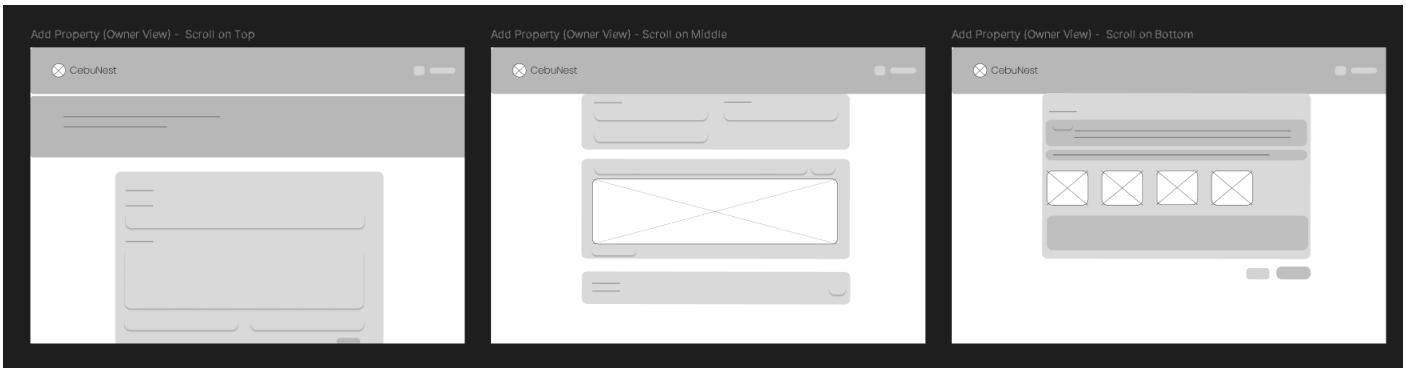
Profile Page and Notification (Tenant)



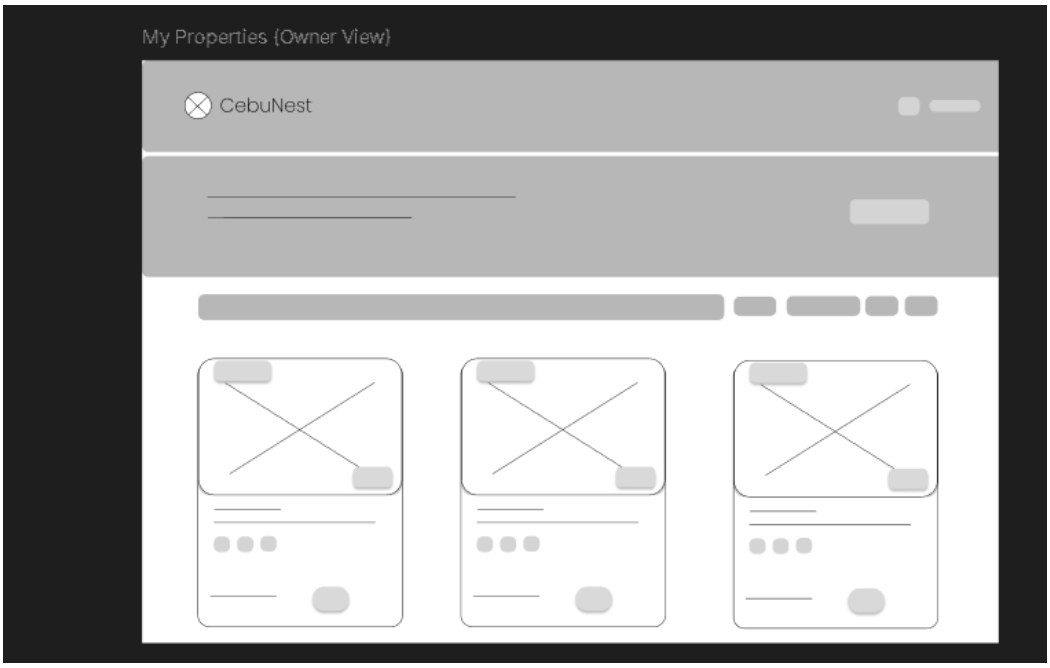
Owner Dashboard (Owner)



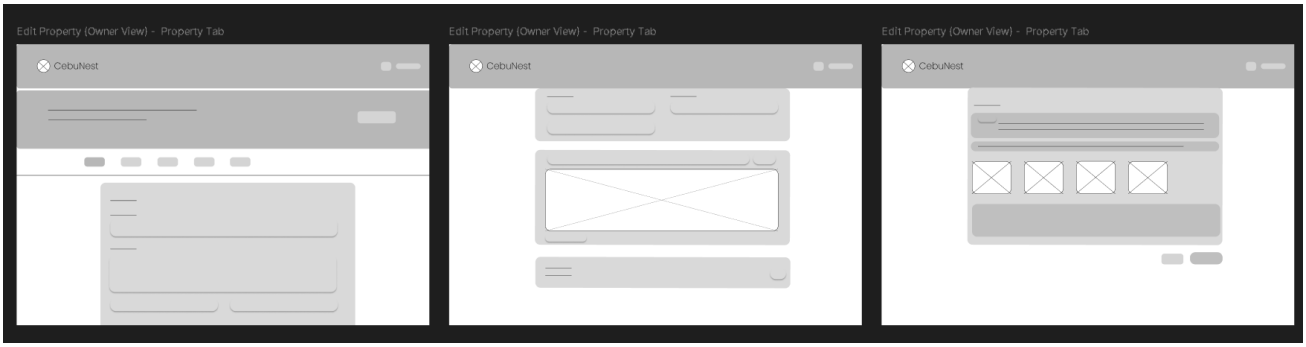
Add Property Page (Owner)



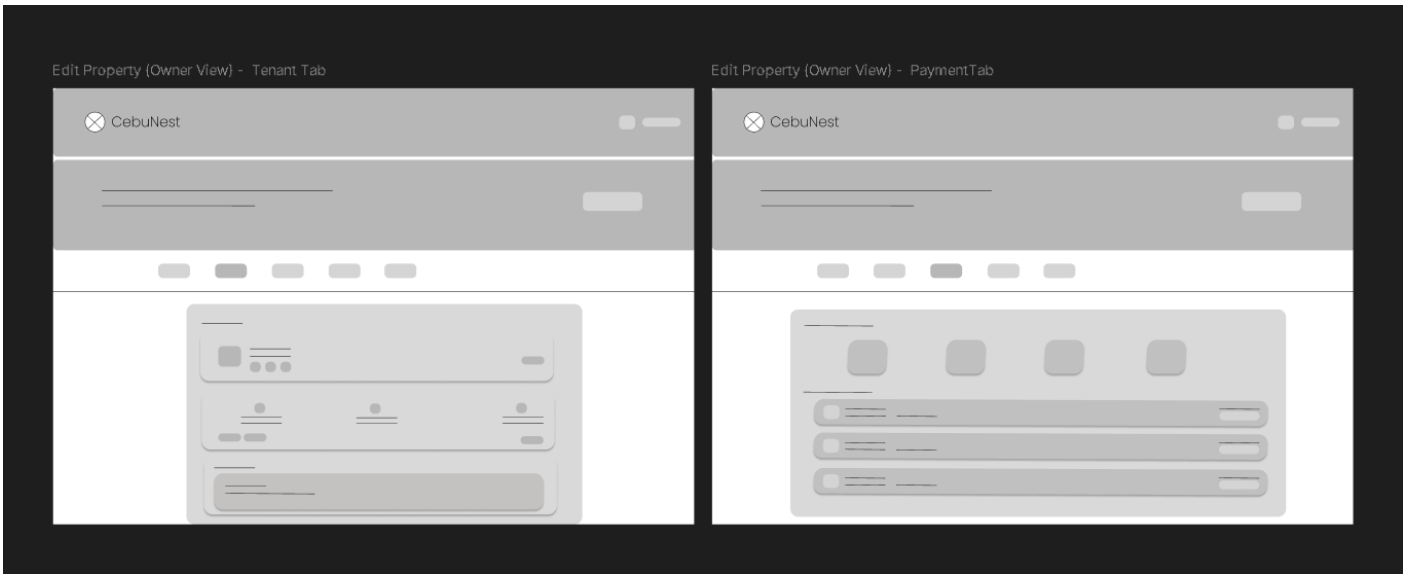
My Properties Page (Owner)



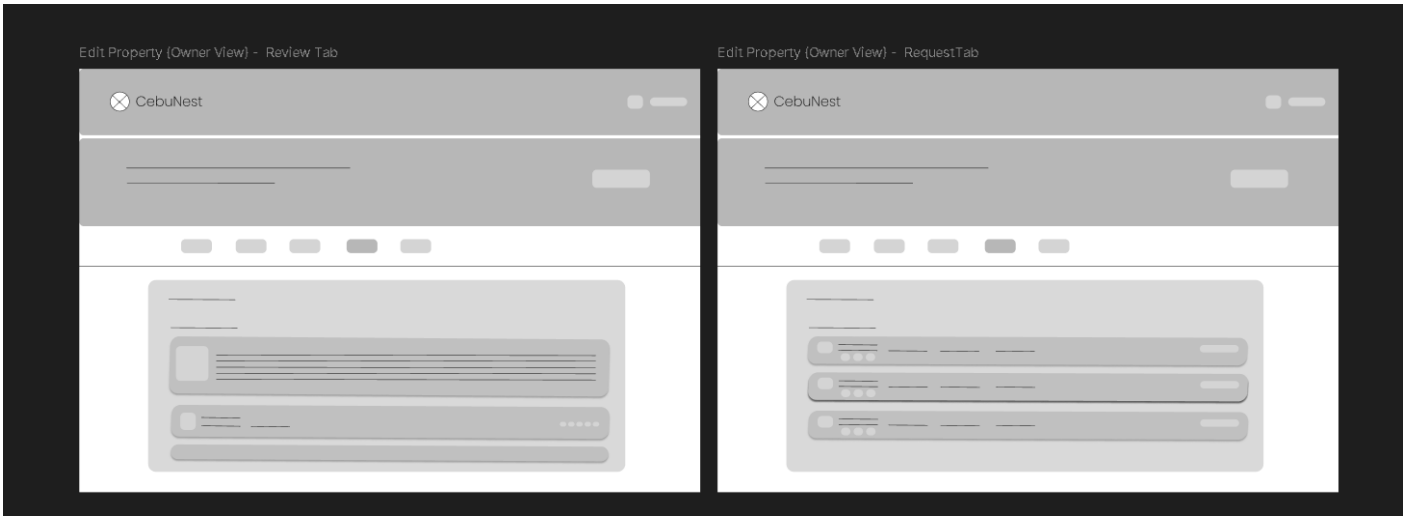
Edit Property Page (Owner) – Property Tab



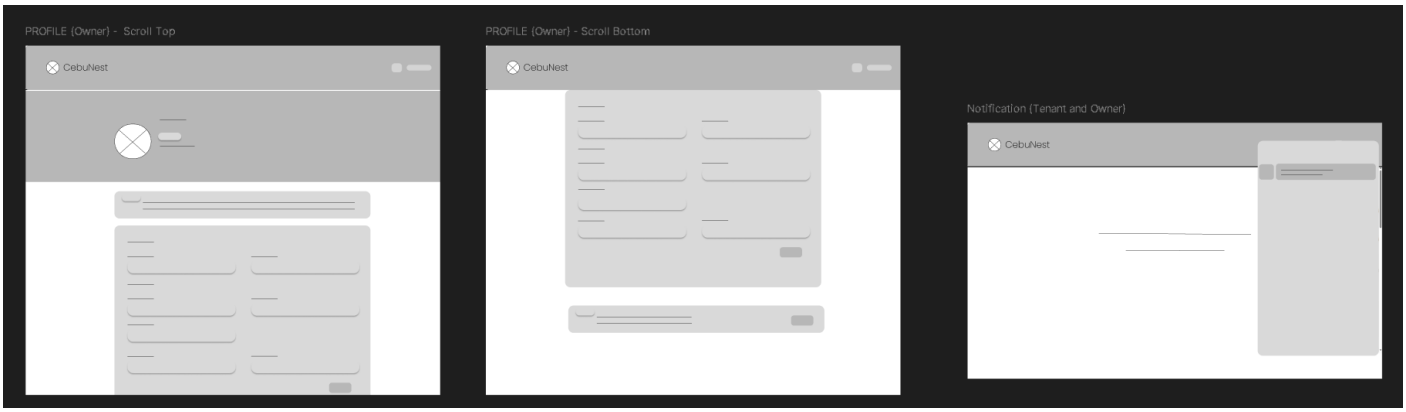
Edit Property Page (Owner) – Tenant and Payment Tabs



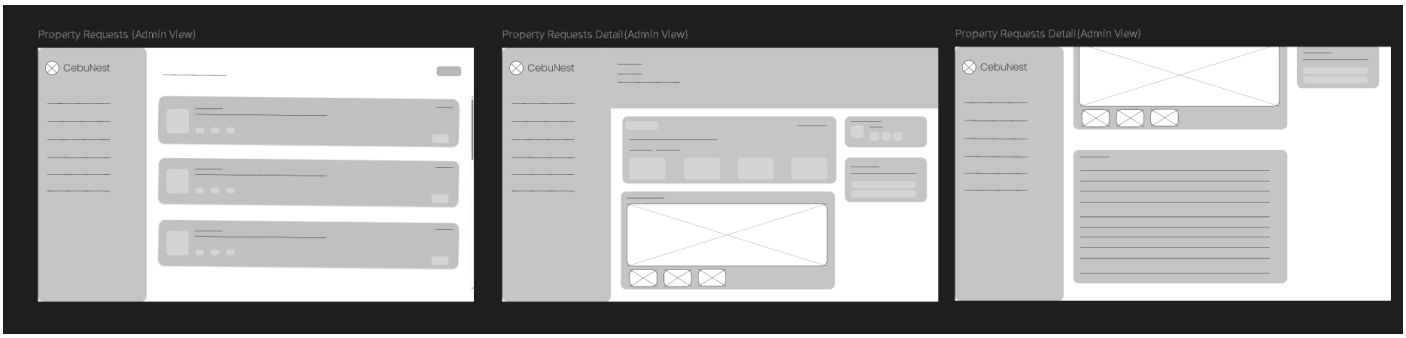
Edit Property Page (Owner) – Review and Request Tab



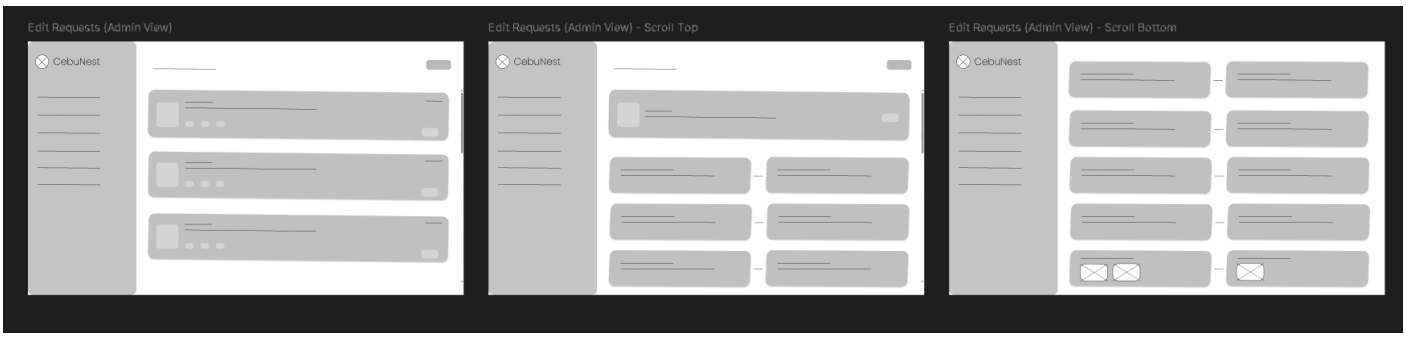
Profile Page and Notification (Owner)



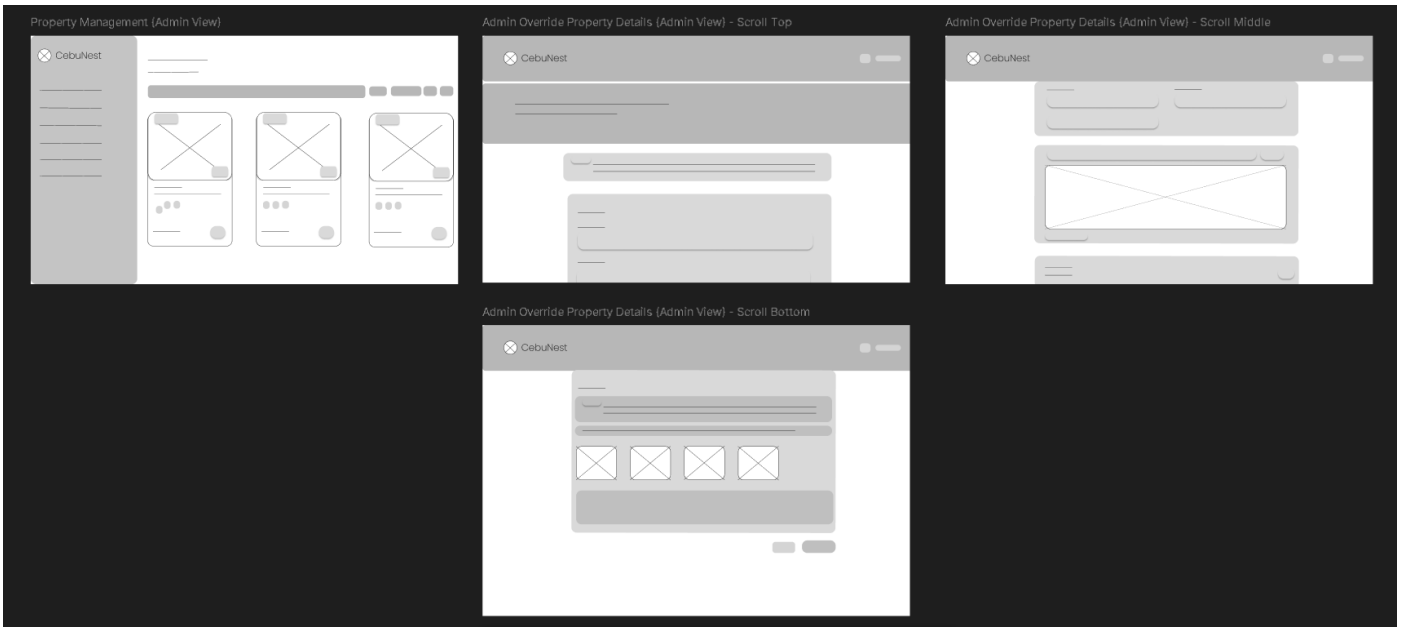
Property Request Pages Flow (Admin)



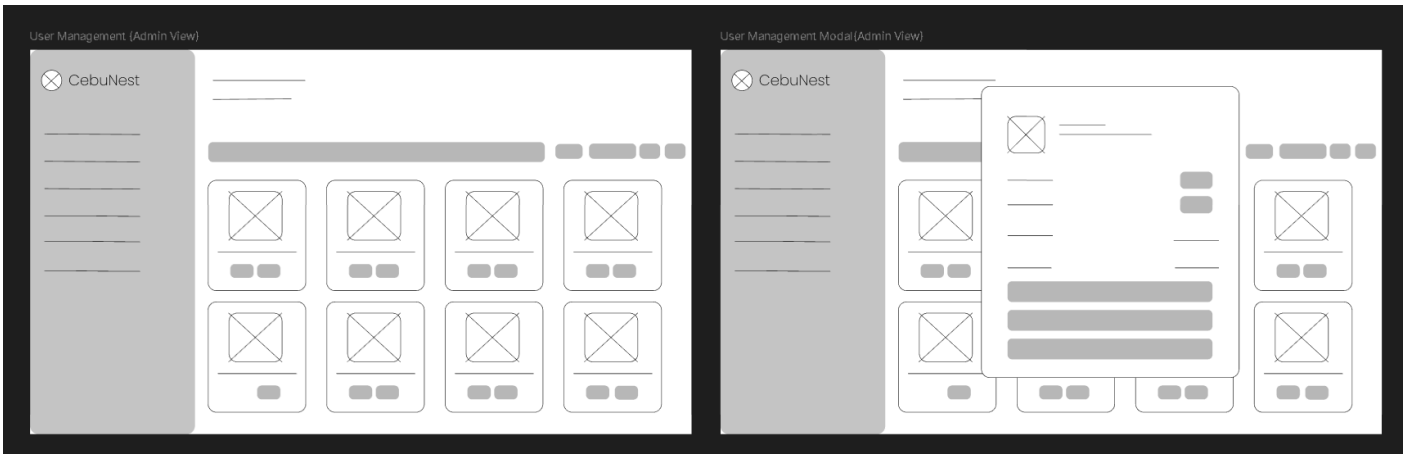
Property Edit Request Pages Flow (Admin)



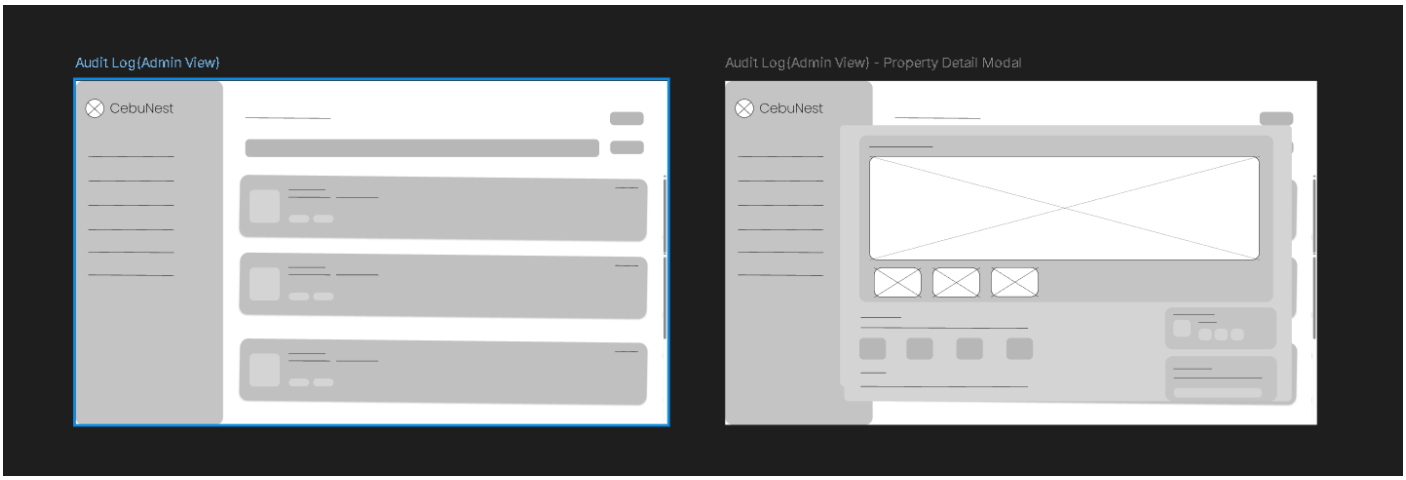
Property Management Pages (Admin)



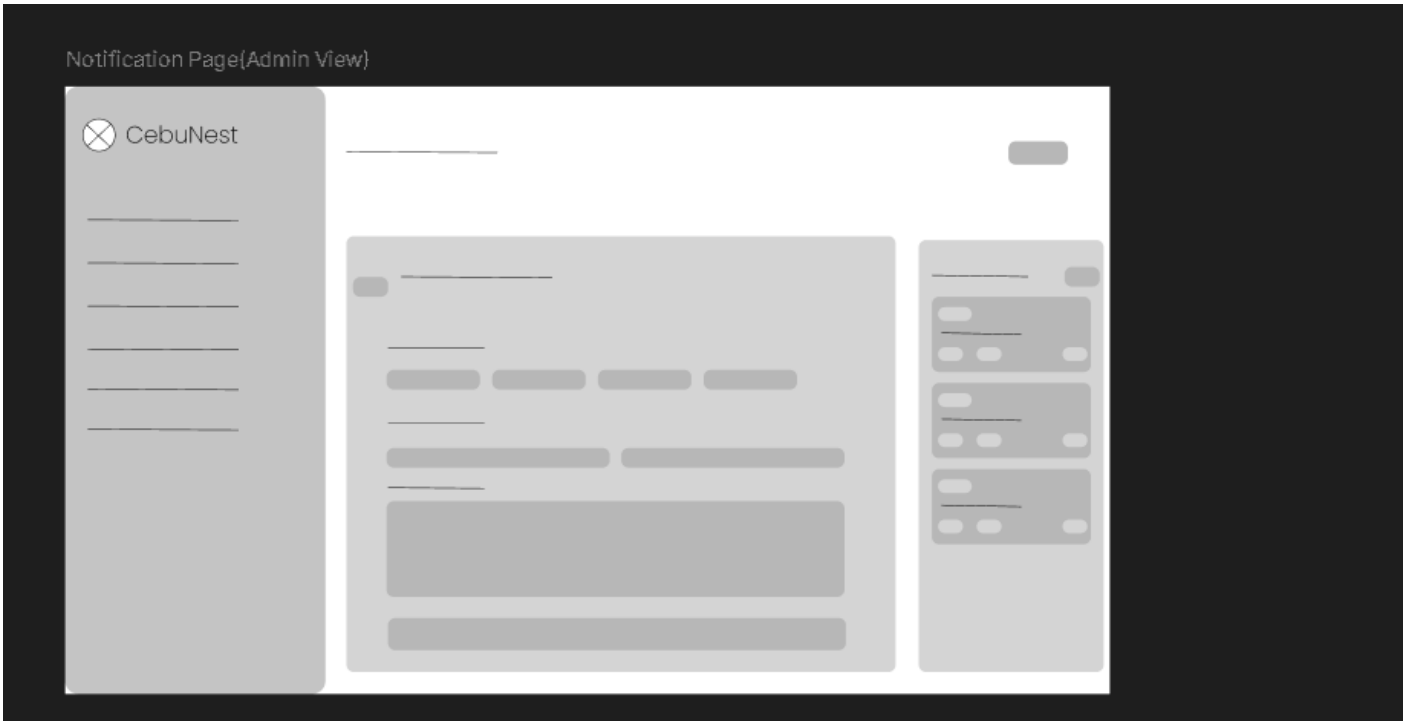
User Management Pages (Admin)



Audit Log Page (Admin)



Notification Page (Admin)

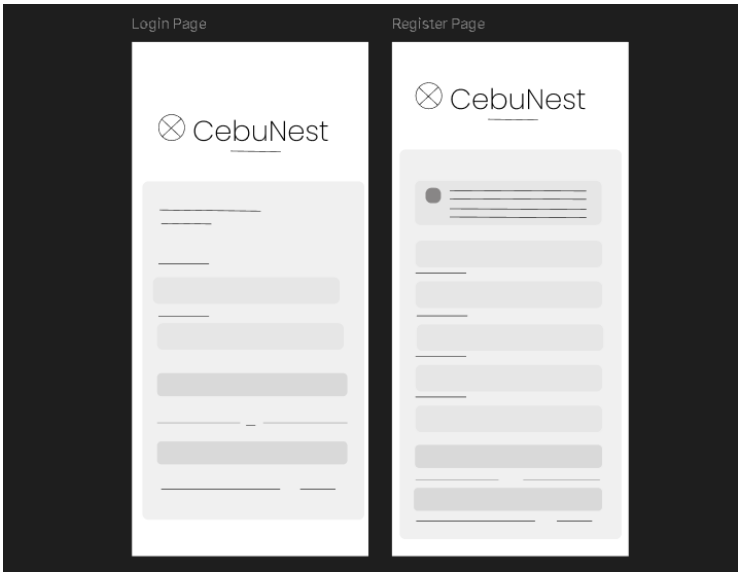


7.2 Mobile Application Wireframes

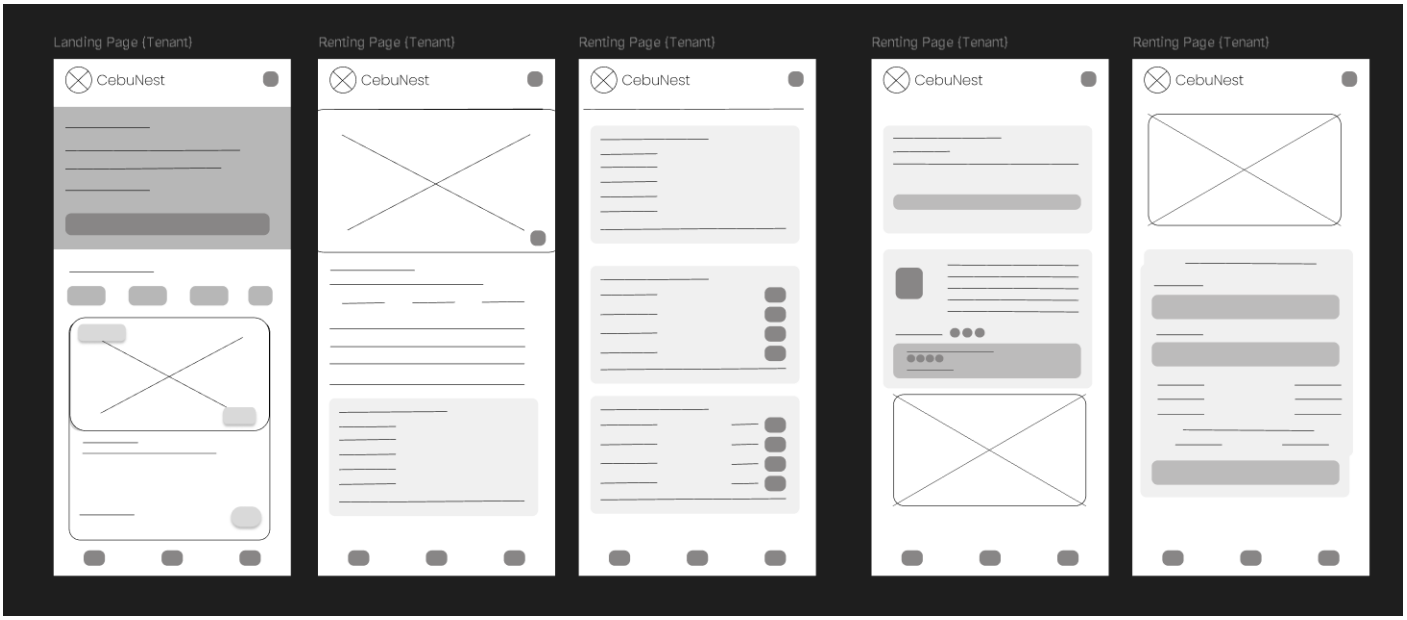
The mobile version of CebuNest will not support admin screens and owner screens; these will be available only on the web version.

Figma Link: https://www.figma.com/design/sxZKvg8tHCKc7PYrDJG9ke/CebuNest-Wireframe?node-id=0-1&t=ekJzqTjnVcs5lALf-1

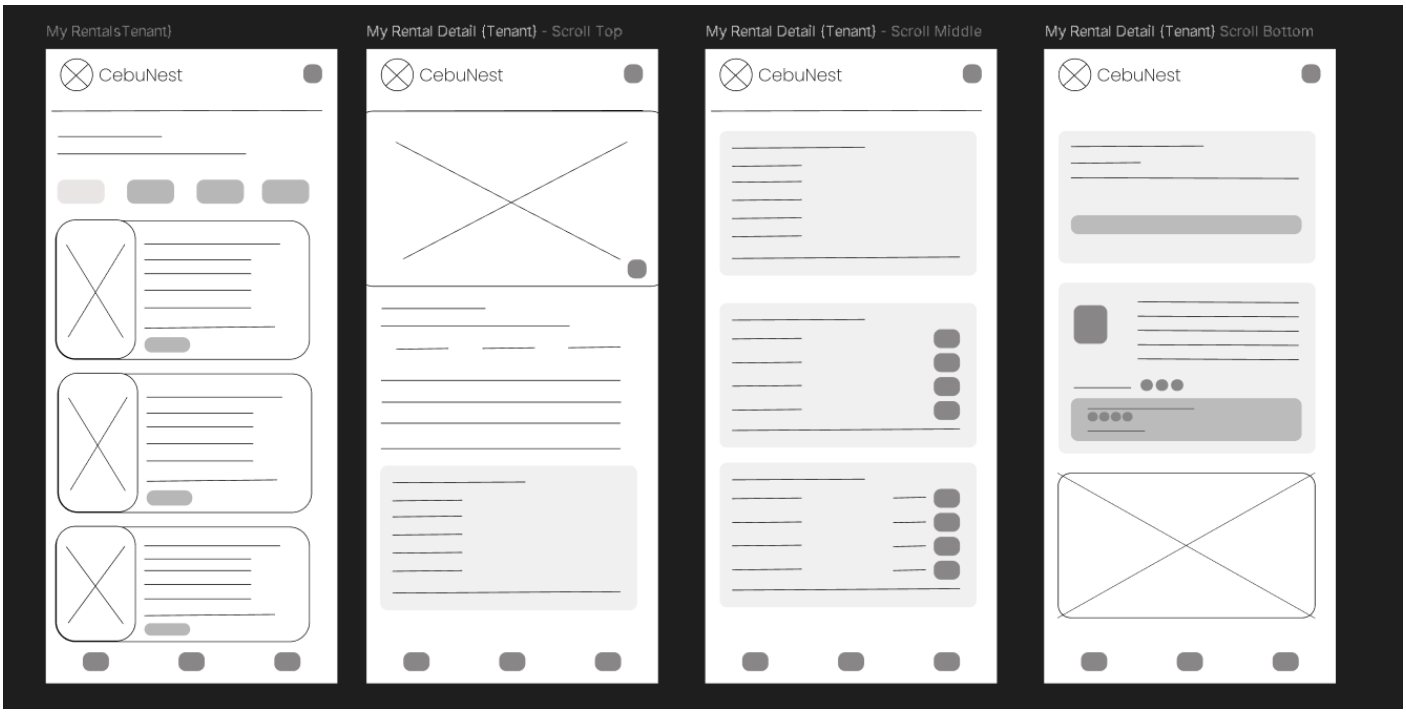
Login Page and Register Page



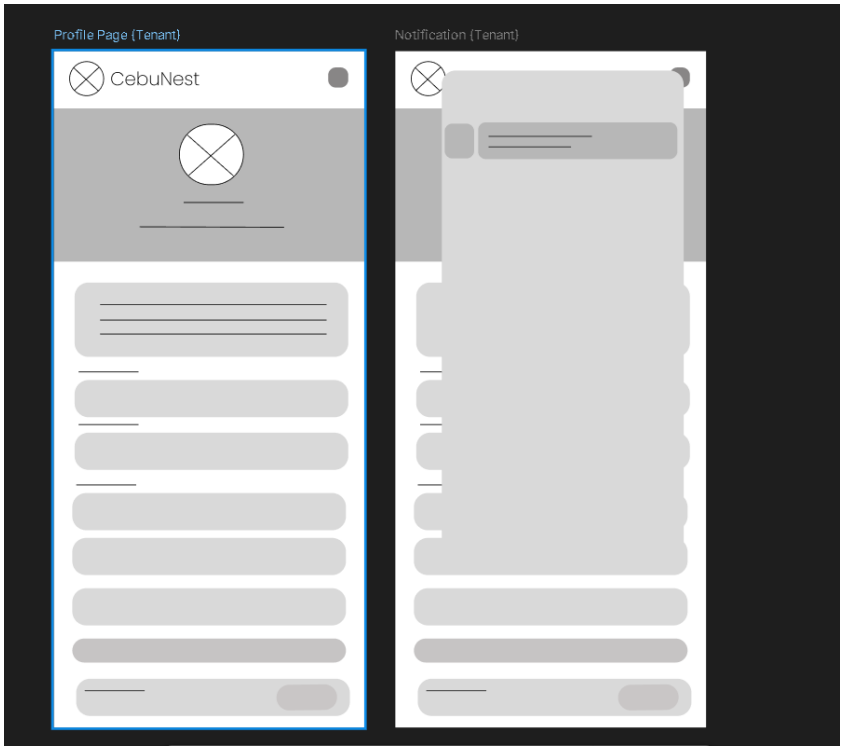
Renting Property Flow Pages (Tenant)



My Rentals Pages (Tenant)



Profile Page and Notification (Tenant)



8.0 PLAN

8.1 Project Timeline Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1–2: Project setup and documentation
Day 3–4: Complete Functional and Non-Functional Requirements
Day 5–7: System architecture design and technology stack confirmation

Week 2: Detailed Design
Day 1–2: API specification and endpoint design
Day 3–4: Database schema and ERD finalization
Day 5–6: UI/UX wireframes in Figma (Tenant, Owner, Admin) Day 7:
Final review of design documents

Phase 2: Backend Development (Week 3-4)

Week 3: Foundation

Day 1: Spring Boot project setup and dependencies
Day 2: Database configuration (PostgreSQL / Supabase) and entity models
Day 3: JWT authentication and role-based access control
Day 4: User management APIs (Tenant, Owner, Admin)
Day 5: Property management APIs (Create, Update, Delete, View)

Week 4: Core Features

Day 1: Rental request system implementation
Day 2: Property approval workflow (Admin review system)
Day 3: Payment integration setup (PayMongo sandbox)
Day 4: Email notification system (SMTP integration)
Day 5: API testing, validation, and documentation

Phase 3: Web Application (Week 5-6)

Week 5: Frontend Foundation

Day 1: React + TypeScript project setup
Day 2: Authentication pages (Login, Register, OAuth login)
Day 3: Property listing page with search and filters

Day 4: Property details page
Day 5: Rental request submission interface

Week 6: Complete Web Features

Day 1: Tenant dashboard (My Rentals, payment status tracking)
Day 2: Owner dashboard (Property management and request handling)
Day 3: Admin dashboard (User management and property approval)
Day 4: Responsive UI improvements and usability testing
Day 5: Frontend integration with backend APIs

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Foundation

Day 1: Android Studio setup and project configuration
Day 2: Authentication screens implementation
Day 3: Property browsing and search features
Day 4: Rental request functionality
Day 5: API service integration using Retrofit

Week 8: Complete Mobile App

Day 1: Tenant rental management and payment status view
Day 2: Owner property management features
Day 3: UI polish and performance improvements
Day 4: Testing on emulator and physical devices
Day 5: APK build generation and documentation

Phase 5: Integration & Deployment (Week 9-10)

Week 9: Integration Testing

Day 1: End-to-end testing across backend, web, and mobile
Day 2: Bug fixing and performance optimization
Day 3: Security review and authentication testing
Day 4: Database and API performance testing
Day 5: Documentation updates

Week 10: Deployment

Day 1: Backend deployment (Railway / Render / Heroku)
Day 2: Web application deployment (Vercel / Netlify)
Day 3: Mobile APK distribution for testing
Day 4: Final system validation

Day 5: Project submission and presentation preparation **Milestones:**

- M1 (End of Week 2): System design and documentation completed
- M2 (End of Week 4): Backend API fully implemented
- M3 (End of Week 6): Web application completed
- M4 (End of Week 8): Mobile application completed • M5 (End of Week 10): Fully integrated and deployed system **Critical Path:**

1. Authentication and role-based access control implementation (Week 3)
2. Property listing and management system (Week 3–4)
3. Rental request workflow (Week 4)
4. Admin property approval system (Week 4)
5. Payment integration and confirmation flow (Week 4–6)
6. Cross-platform system testing (Week 9) **Risk Mitigation:**

- Develop and test core features first (authentication, property listings, rental requests)
- Perform integration testing early to detect API issues
- Maintain version control backups using GitHub
- Monitor API performance and database queries
- Focus on MVP functionality before adding enhancements